

MP1 — Small Language Model Challenge

Student ID: 3036837382
Protocol: 7506-mp1-wt2-v2
Submitted test BPB (full test, FP32 CPU): 1.448065 — validation 1.429294, classroom baseline 2.101260
Frozen predictor: runs/wide512-T1-58k-wd0-s18/checkpoint.pt , sha256
e9deeb465b17b359b55aec2e3537ca53906aa5b26a2ba380f1d55c0c39e38590

Abstract

The task is to train a small GPT from random initialisation and minimise full-test bits per byte (BPB) on the supplied WikiText-2 benchmark, subject to three inference budgets (CPU scoring time $\leq 5\times$ baseline, peak RAM ≤ 4 GiB, uncompressed inference assets ≤ 64 MiB). Training time is unrestricted. Starting from a 2.10126 BPB baseline we reach **val 1.429294 / test 1.448065 BPB**, a reduction of **0.65 BPB (31.1 %)**.

Three findings carry most of the report. First, an early conclusion of this project — that the model was too large for the corpus and that the training-length and model-size axes were "exhausted" — was an **artefact of under-regularisation**: with the identical $6\times$ training budget, only adding dropout 0.1 moved BPB from 1.6936 to 1.5741, and with weight decay the training-length axis reopened. Second, of the four hyper-parameters we swept, **only weight decay and dropout ever mattered**, and both are regularisers; the learning rate was inert in both directions. Third, the single largest lever was not a model change at all: **averaging the predicted probabilities of 21 independently trained models** reaches 1.4142 val BPB — as a number, better than anything any single model of ours achieves. Because the frozen scorer scores exactly one checkpoint, we recover that gain inside one model by **distilling the 21-model probability average into a single student** (1.429294), which is legal, reproducible with the untouched scorer, and costs one model's worth of inference — $3.8\times$ the baseline, comfortably inside the $5\times$ cap — instead of the $21\times$ an ensemble would need.

We also report three clean negative results — weight averaging, log-probability averaging, and temperature-scaled distillation ($T > 1$) — because they are the results that distinguish this report from a list of things that worked.

1. Task and constraints

Train a small autoregressive language model from random initialisation and minimise full-test BPB on the supplied WikiText-2 benchmark with its fixed BPE-2048 tokenizer. Evaluation is fixed: raw WikiText-2 text, independent causal windows of 256 tokens, FP32 on CPU. BPB is the summed negative log-base-2 next-token probability divided by the split's raw UTF-8 byte length.

The asymmetry that shapes every decision below is that **the assignment budgets inference, not training**:

Limit	Value	Where it bites
CPU scoring time	$\leq 5 \times$ baseline	rules out most large models and all ensembles
Peak evaluation RAM	≤ 4 GiB	never binding here (section 7)
Uncompressed inference assets	≤ 64 MiB	binds width at w512 (52 MiB), not w576
Training duration	unrestricted	we used ~ 40 GPU-hours
Data / tokenizer / scorer	fixed, may not be modified	model.py , common.py , evaluate.py , data/ untouched

The training corpus is small: ≈ 3.59 M tokens, i.e. roughly **one token per 300 k parameters** for the models we eventually used. Overfitting, not underfitting, is the dominant risk — and that fact is the plot of sections 4 and 5.

2. Measurement methodology

Five practices were adopted before any improvement was attempted.

(a) A measured noise floor, per regime. Single points cannot be interpreted without one. The same configuration was trained with several seeds at three budgets:

Budget	seeds	validation BPB	σ
1×, width 128	3	2.071088 / 2.073161 / 2.078641 → 2.074297	0.0039
16×, width 256 + dropout 0.1	4	mean 1.539855	0.0030
24×, width 256 + dropout 0.1 + wd 0.3	10	mean 1.516900	0.0080
57.6k distillation steps, width 384, wd 0	2	1.436845 / 1.437540 → 1.437193	0.0005
57.6k distillation steps, width 512, wd 0 (submitted)	2	1.430425 / 1.429294 → 1.429860	0.0008

Working threshold: a single-seed difference below ≈ 0.008 BPB (2σ of the worst regime) is not interpretable; we quote Δ and, wherever possible, a t value. Differences of 0.1 BPB or more — the dropout and ensemble results — are $12\text{--}30\sigma$ and need no such caution.

(b) Matched training budget. Comparisons use the same number of *processed training targets* (steps $\times$ 32 windows $\times$ 256 targets), never the same wall-clock time, so a difference cannot come from one run simply training longer.

(c) Validation-only selection. No configuration, checkpoint or seed was ever chosen with the test split. The test split was scored once for a non-final model only to measure the scoring-time budget (disclosed in section 9); the submitted score is produced once, on the frozen predictor.

(d) Thread-count control. Baseline runs used 4 CPU threads and the rest 2, which would confound the ablation. Re-running the same configuration, seed and step count with 4 vs 2 threads gives 2.0710878209843933 vs 2.0710878177581518 — a thread effect of -3.2×10^{-9} BPB, six orders of magnitude below seed noise (the checkpoints differ; the metric does not).

(e) Cross-device calibration. Work was distributed over four devices, so each was made to re-run the baseline first. All agree to 10^{-6} : 2.0710878209843933 (reference, 4 threads), 2.071083344398206 (AMD Ryzen, 8 threads), 2.0710806613907398 (RTX 4090, scored on CPU), and re-running the same configuration on the reference machine 7 days later reproduced the checkpoint **bit-identically**. Two non-final anchor runs were also replicated across devices (1.574134 vs 1.574273; 1.550847 vs 1.551830) — a paired difference of -0.0233 vs -0.0224 , a direct check that a *large* effect survives a change of device.

3. Direction 1 — architecture modernisation

Four substitutions were made relative to the baseline, leaving the data flow, pre-norm ordering, tied output embedding and causal attention untouched, so that differences are attributable to these four components alone:

Component	Baseline	This work	Claimed benefit
Position	learned absolute embedding	RoPE	relative position is algebra, not parameters
Normalisation	LayerNorm (mean + variance)	RMSNorm (scale only)	one reduction instead of two
MLP	GELU, 4× hidden	SwiGLU , 8/3× hidden	a gate can suppress a feature, not merely fail to activate it
Linear bias	present	absent	redundant after normalisation

All five configurations are within 4 % in parameter count (1,049,216–1,088,256), so no comparison is confounded by size.

Configuration	validation BPB (3 seeds)	Δ	t
baseline	2.074297 $\pm$ 0.003903	—	—
modernised	1.841096 $\pm$ 0.004065	-0.233201	$\approx$ 57
modernised, RoPE $\rightarrow$ learned positions	2.013394 $\pm$ 0.006871	+0.172298	37.4
modernised, SwiGLU $\rightarrow$ GELU	1.931198 $\pm$ 0.004247	+0.090102	26.5
modernised, RMSNorm $\rightarrow$ LayerNorm	1.837418 $\pm$ 0.002068	-0.003678	-1.4

Three observations. **RoPE carries roughly three quarters of the gain** (0.172 of 0.233) — not the component we expected to dominate. **The SwiGLU ablation is parameter-matched** (1,049,728 vs 1,049,216), so its 0.090 is not extra capacity. And **RMSNorm produced no measurable quality effect at all** (t = -1.4): its published motivation is computational efficiency, and that is exactly what we observe. It is kept (cheaper, not worse), but we claim no quality gain from it. The three marginals sum to more than the total (0.262 vs 0.233), so the components interact and their effects are not additive.

4. Direction 2 — longer training, and the under-regularisation trap

Holding the architecture fixed and increasing only the training budget (width 128, no dropout):

Budget	processed targets	validation BPB	Δ vs previous
1×	9.83 M	1.841096	—
3×	29.49 M	1.713145	-0.127951
6×	58.98 M	1.684191	-0.028954
6×, width 256	58.98 M	1.693554	+0.009 (worse than width 128!)

The return fell by 85 % between the second and third points, and the width-256 model at the same budget came out worse than the smaller one. Together with the size sweep of section 5, this produced the project's first conclusion: **the model was too large for the corpus, the length and size axes were exhausted, and further progress would have to come from regularisation**. The first half of that sentence was wrong, and finding out why was the most useful hour of the project.

The dropout experiment. Same configuration, same 6× budget, same seed — one change:

run	dropout	validation BPB
width 256 @ 6×	0	1.693554
width 256 @ 6×	0.1	1.574134

-0.119 BPB from a knob that costs nothing at inference time (dropout is training-only, so neither scoring time nor asset size changes). The "6× penalty" was overfitting, not a data ceiling.

The hyper-parameter sweep. Four knobs, one seed each, width 256 @ 3×:

Knob	validation BPB	Δ vs reference	t	verdict
reference (lr 1e-3, wd 0.1)	1.641426	—	—	—
lr 5e-4	1.653313	+0.008444	+1.61	no effect
lr 2e-3	1.639726	-0.005143	-0.98	no effect
weight decay 0	1.655608	+0.010739	+2.05	marginal
weight decay 0.3	1.620735	-0.024134	-4.60	significant

Confirmed with three seeds: wd 0.1 $\rightarrow$ 1.644869 $\pm$ 0.004542, wd 0.3 $\rightarrow$ **1.624048 $\pm$ 0.004660**, Δ = -0.020821, **t(4) = -5.54**. So the learning rate is inert in both directions while weight decay improves monotonically (0 $\rightarrow$ 0.1 $\rightarrow$ 0.3). Combined with the dropout result, the pattern is unambiguous: **the binding constraint was regularisation, not**

capacity.

Consequences for the axes we had written off. With dropout on, the length axis reopened:

width 256, dropout 0.1	validation BPB
12×	1.542373
16× (4 seeds)	1.539855 ± 0.0030
20×	1.533100
24×	1.531583
32×	1.529183

and with wd 0.3 stacked on top of it (single seeds, seed 17):

width 256, dropout 0.1 + wd 0.3	validation BPB	Δ vs previous
12×	1.528085	—
16×	1.520393	−0.007692
24× (4 seeds)	1.517227 ± 0.0049	−0.003166
32×	1.510043	−0.007184
48×	1.503535	−0.006508

The last two steps (−0.0072, −0.0065) are 1.5–2.3 σ : the axis is flattening, not yet flat. Two smaller sweeps complete the picture: dropout 0.15 is indistinguishable from 0.10 at 9× (1.551268 vs 1.550847) and slightly better at 16× (1.529514 vs 1.537587), while dropout 0.2 is clearly worse at both (1.575391, 1.548067); batch 64 instead of 32 at 16× is worse (1.550404 vs 1.537587); warmup 500 instead of 100 is worse (1.544672 vs 1.542373). **Our best plain single model is width 256 @ 48× with dropout 0.1 and wd 0.3: 1.503535 val BPB**, down from the 2.074297 baseline — and the honest lesson is that almost all of it came from fixing regularisation rather than from adding capacity or training.

5. Direction 3 — scale, revisited under proper regularisation

The size sweep that produced the "too large for the corpus" conclusion was run in the under-regularised regime, so it is not evidence about scale in general. Repeating it later — and in the distilled students of section 6, which are trained on a regularised teacher's soft targets rather than on hard labels — **the sign of the width effect flips**:

width sweep	regime	w256	w320	w384	w448	w512
(a) width 128/192/256/320, 3×, no dropout	under-regularised	1.641426	1.642841	—	—	—
(b) width 320/384/448, 12×, dropout 0.1	under-regularised	—	1.560085	1.577331	1.593073	—
(c) distilled students, 43k steps, wd 0	regularised targets	1.449643 ¹	1.446941	1.438438	—	—
(d) distilled students, 57.6k steps, wd 0 (submitted recipe)	regularised targets	1.461738	—	1.436845	1.433143	1.430425

¹ width 256 **depth 6**, the parameter-matched comparison (5.25 M vs 3.67 M params); at the same length the depth-4 width-256 student is 1.461738. ² every column of row (d) is seed 17, so the row is a paired comparison; the submitted checkpoint is seed 18 of the width-512 cell (validation 1.429294, test 1.448065, selected between the two seeds on validation only).

In (a) and (b) width is neutral-to-harmful — that is the plateau that misled us. In (c) and (d), which are properly regularised, width improves monotonically, and the gain is large enough to matter: 256 → 384 is −0.025 BPB. **The same lever has opposite signs in the two regimes**, which is why the original conclusion was not merely imprecise but wrong in direction.

In (d) the axis stays open all the way to the widest model the budget allows: 384 → 448 is −0.0037 and 448 → 512 a further −0.0027. **It ends at the asset limit, not at diminishing returns**: width 576 would be 17,110,080 parameters, i.e. a 65.28 MiB checkpoint against a 64 MiB cap, so width 512 — 52.02 MiB, 3.75× the baseline scoring time — is the widest model that can be submitted at all. Two seeds at the submitted width (1.430425 / 1.429294) and two at width 384 (1.436845 / 1.437540) agree to 0.0008 and 0.0005 BPB, so those width differences are 3–8× the seed spread of the same recipe.

The depth axis, in contrast, is genuinely without leverage once parameters are matched (all $|t| < 1$): wide256 (3,673,344 params) 1.644869 ± 0.0045, deep224×5 (3,470,656) 1.641882, deep192×7 (3,492,672) 1.647380. With head_dim fixed at 32, shape is freely exchangeable at this scale; only total parameter count matters.

The width-448 point was not run at the shorter 43k budget; the 43k row of the table is used only to show the direction of the trend, and every number in row (d) is measured at the submitted length.

6. Direction 4 — ensembling, and distilling an ensemble into one checkpoint

6.1 Probability averaging

Averaging the *predicted distributions* of M independently trained models is worth far more than any architecture change we made:

Pool	members	member mean	probability average	gain
width 256, 16×, dropout 0.1	2	1.5398	1.483437	−0.056
	3	1.5398	1.465464	−0.074
	4	1.5398	1.455747	−0.084
width 256, 24×, dropout 0.1 + wd 0.3 (best recipe)	15	1.516424	1.418004	−0.098
	21	1.516452	1.414189	−0.102

The rule matters. For logits z_m over the 2048-token vocabulary the ensemble predicts $p_T(v | x) = (1/M) \cdot \sum_m \text{softmax}(z_m(x))_v$ — an arithmetic mean of **probabilities**, computed as `logsumexp` over per-model log-softmax outputs. Averaging log-probabilities (a geometric mean, the easier implementation) is **provably identical to the member mean**: the recorded runs give 1.5164235326 for the log-prob average against 1.5164235325 for the arithmetic mean of the same 15 members — a difference of 1×10^{-10} . Geometric averaging is not a weaker ensembling method; it is not ensembling at all.

6.2 Why we cannot just submit the ensemble

21 members would cost ≈ 21× the baseline scoring time (limit 5×) and the frozen scorer in `evaluate.py` takes exactly **one** checkpoint — so an ensemble score could not be reproduced from the submitted artefact, and an unreproducible number is indistinguishable from a mis-reported one. Both constraints point the same way: get the ensemble's benefit into one model.

6.3 Distillation: the ensemble, compressed into one checkpoint

We train a single student on the ensemble's soft targets. For a batch of windows,

$$L = - \sum_v p_T(v | x) \cdot \log p_S(v | x)$$

which is $KL(p_T || p_S) + H(p_T)$, and the entropy term $H(p_T)$ does not depend on the student, so minimising L is exactly minimising the divergence from the ensemble. Teachers are our own checkpoints, trained from scratch on the provided training split; the soft targets are computed on **training** windows only, and the validation split is used solely to score the resulting student. No test data, no external text and no pretrained weights enter anywhere.

The result: **the student beats 2- and 3-member ensembles** (1.436845 vs 1.483437 and 1.465464) at 1× inference cost, so the "is it worth the risk of submitting an ensemble?" question disappears — we submit a single ordinary checkpoint.

6.4 The six levers, one variable at a time

Lever	Reference → variant	Δ validation BPB
student dropout 0.1 → 0	1.517886 → 1.482734	−0.0352
student capacity w256 → w384 (43k steps)	1.468425 → 1.442600	−0.0258
student steps 14.4k → 57.6k (w256, wd 0)	1.482734 → 1.461738	−0.0210
student wd 0.3 → 0 (28.8k steps)	1.472267 → 1.467645	−0.0046
student lr 1e-3 → 5e-4 (28.8k steps, wd 0)	1.467645 → 1.475274	+0.0076 (worse)
temperature T = 1 → 4 (28.8k steps)	1.472267 → 1.515132	+0.0429 (worse)

Two deserve comment. The **temperature** result contradicts the textbook recipe, which softens the target so the student can see the tail; our T = 4 run is 0.043 BPB worse than T = 1, because our objective is already a log-loss — at T = 1 the cross-entropy **is** the NLL, the exact quantity the benchmark measures — so softening the target trades head accuracy for a tail the metric barely scores. Hinton's temperature is a device for objectives that are not already log-loss. The **dropout** lever means the student must be de-regularised: it fits a smooth averaged distribution, so the noise that rescued the plain models (section 4) now only adds variance.

6.5 A third negative result: weight averaging

Averaging the *weights* of four runs of the same configuration but different training lengths (the standard "model soup" trick) gives 2.076 BPB — essentially the baseline, i.e. catastrophic, because the endpoints are far apart in weight space. Weight averaging is only meaningful between near-identical runs. We report it because it is cheap to try and easy to get wrong: on this task the safe version of "combine several models" is probability averaging, and the safe way to serve it is distillation.

7. The frozen predictor and the three inference budgets

7.1 Frozen configuration

Component	Value
Architecture	student.py : pre-norm block, RMSNorm, SwiGLU, RoPE, no linear biases
Width / heads / depth	512 / 16 / 4 (head_dim 32)
Parameters	13,634,048
Student training	57,600 steps × 32 × 256 targets = 4.72×10^8 targets, lr 1e-3, warmup 100, batch 32, wd 0, dropout 0; two seeds run (17, 18), seed 18 selected on validation
Teachers	21 checkpoints (16 seeds of width 256 @ 24× + dropout 0.1 + wd 0.3, plus 12×/16×/32×/48× and a dropout-0.15 variant)
Distillation	probability-averaged soft targets, T = 1, pure distillation (hard-label weight 0)

7.2 Measured budgets

All timings were measured on an idle machine, validation split, 4 CPU threads (the thread count of the course's own reference measurement), with the baseline re-measured in the same session as a control and every model measured twice:

Model	Parameters	Assets	Scoring time	× baseline	Peak RSS
baseline	1,088,256	4.17 MiB	8.35 s (9.22 / 7.47)	1.00	1.67 GiB
width 256	3,673,344	14.02 MiB	14.95 s	2.00	1.66 GiB
width 320	5,572,160	21.27 MiB	16.99 s	2.27	1.67 GiB
width 384	7,867,776	30.03 MiB	20.26 s	2.71	1.71 GiB
width 448	10,557,120	40.28 MiB	23.28 s	3.11	1.74 GiB
width 512 (submitted)	13,634,048	52.02 MiB	27.94 s	3.74	1.77 GiB
width 576 — assets over the cap	17,110,080	65.28 MiB	32.47 s	4.34	1.89 GiB
budget limit	—	64 MiB	37.4 s (5×)	5.00	4 GiB

Ratios are taken against the *fastest* baseline of the session, which is the conservative choice: the baseline itself repeated to 9.22 s and 7.47 s within ten minutes ($\pm 10\%$). Measured on the **test** split in the session that produced the submitted score, the same ratio is **3.77×** (baseline 7.93 s, submitted 29.90 s), and across every budget session the submitted shape measured 3.7×, 3.8× and 3.9×. The submitted model therefore uses about three quarters of the time allowance, 81 % of the asset allowance and 44 % of the memory allowance.

The width-320/448/512/576 rows are randomly initialised *shape probes*: scoring time and memory depend on the shape, not on what the weights contain, so a shape can be priced before it is trained. Width 576 was rejected on that basis without spending a single GPU-hour.

7.3 What the cost curve actually looks like

Scoring time grows **sub-linearly** in width: quadrupling it (128 → 512) costs 3.3× the time — not 4×, and certainly not the 16× that width-squared FLOPs would suggest — because a fixed part of the work, the 2048-way output projection and the normalisation, per position, does not scale with width at all. That is why parameter-count extrapolation misestimates this budget, and why an earlier estimate of ours (width 384 at 4.10×, taken under background load) was wrong: the controlled figure is 2.7×. The same effect makes the 4-thread setting the *worst* case rather than the best — width 512 measures 3.74× at 4 threads but only **2.99×** at 1 thread, because the baseline loses more than the wide model when threads are added. Peak RAM barely moves with width at all (1.67 → 1.78 GiB while the weights grow 4 → 52 MiB), because it is dominated by the loaded corpus rather than by the model.

7.4 Final score

	validation BPB	test BPB	scoring time (test split, 4 threads)
baseline	2.071083	2.101260	7.93 s
submitted predictor (width 512, seed 18)	1.429294	1.448065	29.90 s (3.77×)

The test number is produced by the unmodified scorer on the frozen checkpoint: `python evaluate.py --checkpoint <ckpt>.pt --device cpu --precision fp32 --split test`. The checkpoint is selected on the **validation** column only (section 2c). For completeness, the validation and test scores of every candidate are:

candidate (all 57,600 steps, wd 0, T = 1)	validation BPB	test BPB
width 512, seed 18 — submitted	1.429294	1.448065
width 512, seed 17	1.430425	1.449404
width 448, seed 17	1.433143	1.452505
width 384, seed 17	1.436845	1.457566
width 384, seed 18	1.437540	1.455725

Selection used the validation column; the test column is reported for every candidate so that the reader can see the selection transfer (the validation and test orderings are identical). The non-selected seeds' test scores were computed after the configuration was frozen and were not used to choose anything.

8. What was not attempted

- **Optimiser replacement** (e.g. Muon). A high-variance change to implement and debug; the learning-rate sweep showed the optimiser settings we have are not the binding constraint.
 - **Born-again / iterative distillation** (distilling from the ensemble *plus* an earlier student). Cheap and literature-supported, but cut in favour of the paired width runs, which had a measured monotone trend behind them.
 - **Enlarging the teacher pool**. Teachers cost ~15 min of GPU each and eight more seeds were affordable; the ensemble curve (section 6.1) is visibly flattening by 21 members, so the expected student gain did not justify the GPU hours.
 - **Submitting the ensemble** — not available: the frozen scorer takes one checkpoint, so an ensemble number could not be reproduced from the submitted artefact. Under the corrected budget of section 7.2 a two-member ensemble *would* have fitted inside 5×; the single distilled checkpoint was preferred anyway, because it is reproducible with the official scorer.
 - **Training on train + validation** for the final model. Legal but it destroys the selection signal and would make the reported validation number meaningless.
 - **Long-context / sparse attention** (GQA, MLA, sliding window). They reduce KV-cache and long-sequence serving cost; this protocol scores independent 256-token windows with no cache.
-

9. Critical analysis and limitations

The most useful negative result is that our first conclusion was wrong. "The training-length and size axes are exhausted" came from runs with no regularisation, and adding dropout 0.1 at a fixed budget moved BPB by $-0.119 - 30\sigma$. The general lesson: when a scaling curve turns over on a small corpus, the first hypothesis to test is overfitting, not capacity. We state this at length because the same mistake is invisible in a final report that only shows the working recipe.

A component's published benefit may be about cost, not quality. RMSNorm's effect here is 0.004 BPB with $t = -1.4$, and the bundle's total effect (0.233) exceeds the sum of its parts.

Temperature is not universal. The Hinton-style temperature exists to make a non-log-loss objective informative; our objective is already the benchmark's log-loss, and $T > 1$ cost us 0.043 BPB. We report the failed setting rather than silently using $T = 1$.

Ensembling is the largest effect and the least reproducible artefact, which is precisely why we distilled it. But the student is not the ensemble: 1.429294 against the 21-member 1.414189. Measured against the ensemble's own member mean (1.516452), the student captures **85 %** of the distance from an average teacher to the full ensemble — while being one model instead of 21. The residual 0.015 BPB is a compression loss: a single 13.6 M-parameter model cannot represent the average of 21 models of 3.67 M, and that gap sets the ceiling on this approach at this budget.

Cost measurements are fragile. An earlier estimate of ours put width 384 at 4.10× — measured with background load on the machine — where the controlled measurement is 2.8×; the submitted width-512 model measures 3.8× on both splits. All budget numbers here were re-measured on an idle machine in one session, with the baseline included as a control.

Single-seed points remain in the length, dropout, batch, warmup, temperature and lr sweeps. Their differences are several times the noise floor so their directions are reliable, but exact values are not. Both configurations that matter are multi-seeded: two seeds at the submitted width (spread 0.0008 BPB) and two at width 384 (0.0005), which is what makes the width differences of section 5 readable.

Disclosure of an earlier test-split scoring. One interim test-split evaluation was run on a non-final model solely to measure the scoring-time budget. It influenced no selection decision; the score reported in section 7.4 is computed once, on the frozen predictor, after the configuration was locked.

10. Reproduction

REPRODUCE.md in the submitted repository contains the complete sequence: environment installation, the sanity tests, the exact training and distillation commands, and the budget measurements. Scoring the submitted number requires only the checkpoint bundle — **no retraining**:

```
cd code
python evaluate.py --checkpoint <frozen-checkpoint>.pt \
    --device cpu --precision fp32 --split test
```

This prints the submitted BPB. Nothing in `model.py`, `evaluate.py`, `common.py`, `data/` or `tests/` was modified; `train.py` gained three *optional* flags (`--lr`, `--weight-decay`, `--warmup`) whose defaults reproduce the original recipe exactly, so the documented baseline command still behaves as documented.

Cost of full reproducibility, stated honestly. The distillation pipeline needs the 21 teacher checkpoints (≈ 310 MiB, shipped as a separate archive) plus a GPU: 57,600 distillation steps at ≈ 110 ms/step is ≈ 1.8 GPU-hours per student. On CPU the same run is ~ 15 s/step, i.e. about ten days, so CPU retraining of the final model is not a realistic instruction — and not required: the checkpoint is the artefact. A CPU smoke test of the same code path is provided instead (`distill.py` on two small local teachers, 20 steps, ≈ 25 s), which exercises the objective, the teacher loading and the checkpoint format end to end on ordinary hardware.

11. AI assistance disclosure

Disclosed as required by the assignment:

- **AI-assisted, and disclosed as such:** environment setup; the experiment runner and queue scripts; the implementation of RoPE/RMSNorm/SwiGLU in `student.py` (the author verified it against the provided correctness tests, which pass); `distill.py`, `ensemble_eval.py`, `soup.py`, `select_final.py`, `measure_budget.py`, `budget_table.py`, `_peak_wrap.py`; the execution and bookkeeping of the sweeps; and the structure and tables of this report. The author set the scope, made every method decision from the measured data, and reviewed and corrected the final text; the argument and the interpretation of the results are the author's.
 - **Not AI-generated:** every number in this report comes from a run recorded in `code/run_log.csv`, in the per-run `metrics.json` files, or from the budget measurements described in section 7. No result was invented, extrapolated or estimated, and no AI system produced any experimental measurement.
 - No pretrained weights, external training text, retrieval index or test-set tuning was used. The data, tokenizer and evaluator were left unchanged.
-

References

1. Raschka, S. (2024). *Build a Large Language Model (From Scratch)*. Manning.
2. Vaswani, A. et al. (2017). Attention Is All You Need. arXiv:1706.03762.
3. Su, J. et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv:2104.09864.
4. Shazeer, N. (2020). GLU Variants Improve Transformer. arXiv:2002.05202.
5. Zhang, B. & Sennrich, R. (2019). Root Mean Square Layer Normalization. arXiv:1910.07467.
6. Merity, S. et al. (2016). Pointer Sentinel Mixture Models. arXiv:1609.07843.
7. Hinton, G. et al. (2015). Distilling the Knowledge in a Neural Network. arXiv:1503.02531.
8. Furlanello, T. et al. (2018). Born-Again Neural Networks. ICML.
9. Wortsman, M. et al. (2022). Model Soups. ICML.